

演算法總覽

Algorithm Catalog · 串連系統 v2.1

本文件完整列出串連系統使用的 22 種演算法 / 模型，每筆含核心公式、實作檔案位置、與選用理由。涵蓋資訊檢索、統計分析、時間序列、加權評分、圖論、狀態判定、預測模擬等領域。

① 資訊檢索	BM25F、RSJ IDF、多 n-gram、同義詞、Substring Boost	6
② 統計分析	Empirical Percentile、Gini Coefficient、敘述統計	3
③ 時間序列	Exponential Decay、Local Minima、asOf Snapshot、Weekly Series	4
④ 加權評分	Weighted Load、ORI、Org Health、Decision Impact、Leader Score	5
⑤ 圖論網絡	Adjacency Matrix、Force-directed、Asymmetric Detection	3
⑥ 狀態判定	Decision Helpers、Risk/Load/Health Level	2
⑦ 預測模擬	What-if Simulation、Impact Prediction	2

① 資訊檢索演算法 (Information Retrieval)

#1 BM25F — Okapi BM25 with Field Weighting

資訊檢索 · 核心

公式 / 邏輯

$$\text{score}(q,d) = \sum \text{idf}(t) \times \sum \text{field_weight}(f) \times \text{tf_norm}(t,d,f)$$
$$\text{tf_norm} = \text{tf} \times (k_1 + 1) / (\text{tf} + k_1 \times (1 - b + b \times \text{len} / \text{avgLen}))$$
$$k_1 = 1.5, b = 0.75$$

實作位置

src/lib/historySearch.ts — buildIndex(), searchHistory()

說明

Elasticsearch / Lucene 同款。比 TF-IDF cosine 更接近人類認知 (TF 飽和) ; 比 LLM Embedding 可解釋、零成本、保護機密。中小企業 50-200 筆歷史案的最佳選擇。

#2 Robertson-Sparck-Jones IDF

資訊檢索 · 輔助

公式 / 邏輯

$$\text{idf}(t) = \log(1 + (N - \text{df}(t) + 0.5) / (\text{df}(t) + 0.5))$$

實作位置

src/lib/historySearch.ts — buildIndex()

說明

BM25 標準的 IDF 變體, +0.5 是 Lidstone smoothing, 避免極端值。對 stop word (的、了、是) 給 0 分, 對罕見詞給高分。

#3 多 n-gram 中文 Tokenization

資訊檢索 · 前處理

公式 / 邏輯

■■■■■■■■■ → ■/■/■/■/■ (1-gram)
→ ■■/■■/■■/■■/■■ (2-gram)
→ ■■■/■■■/■■■/■■■ (3-gram)

實作位置

src/lib/historySearch.ts — tokenize()

說明

中文無詞界。1+2+3-gram 混合切詞, 可同時抓單字、雙字詞、3 字專有名詞。投資領域多為 3-4 字術語 (投委會、董事會、伊勢島飯店), 3-gram 特別重要。

#4 Substring Boost

資訊檢索 · 增強

公式 / 邏輯

```
if document.lower().includes(query) → score × 1.8
if title.lower().includes(query) → score × 1.4
```

實作位置

src/lib/historySearch.ts — searchHistory()

說明

解決 n-gram 把專有名詞拆碎後被稀釋的問題。例如搜「東京中央銀行」，光靠 n-gram 命中分數會被「銀行」這種 high-DF 字拉低；substring boost 確保完整匹配優先。

#5 Synonym Normalization（同義詞表）

資訊檢索 · 前處理

公式 / 邏輯

```
14 tokenize canonical
   ~ ~ fundraising ~ ~ DD
NDA ~ ~ IC ~ ~ exit ...
```

實作位置

src/lib/historySearch.ts — SYNONYM_MAP, normalizeSynonyms()

說明

金融術語經常有中英文 / 縮寫多種寫法，正規化提升命中率。從長到短替換，避免子字串覆蓋（先替「盡職調查」再替「盡調」）。

#6 Cosine Similarity（已退役，保留說明）

資訊檢索 · 歷史

公式 / 邏輯

$$\text{sim}(\mathbf{q}, \mathbf{d}) = (\mathbf{q} \cdot \mathbf{d}) / (||\mathbf{q}|| \times ||\mathbf{d}||)$$

實作位置

v1 用 TF-IDF + Cosine（已被 BM25F 取代）

說明

v1 採用過。線性 TF 計分讓「出現 5 次」與「出現 50 次」差距過大，且無 field weighting。v2 升級到 BM25F 後，cosine 不再用於主流程。

② 統計分析 (Statistics)

#7 Empirical Percentile (經驗分位數)

統計 · 核心

公式 / 邏輯

```
percentile(arr, p):  
    sorted = sort(arr)  
    idx = (n-1) × p/100  
    return linear_interp(sorted[floor(idx)], sorted[ceil(idx)])
```

實作位置

src/lib/algorithms.ts — stats.percentile()

說明

用「同類歷史解決天數分布」評估當前卡點風險，比絕對天數門檻更精準。不同類別合理時長差異極大（法遵 7-8 天、跨部門 4-5 天），percentile 自動適應。

#8 Gini Coefficient (不平均度, 經濟學標準)

統計 · 核心

公式 / 邏輯

```
Gini =  $\frac{\sum (2i - n - 1) \times \text{score}[i]}{n \times \sum \text{score}[i]}$   
0 = 0.0000001 = 0.35 0.000000
```

實作位置

src/lib/algorithms.ts (computeORI), src/lib/orgHealth.ts (loadBalance)

說明

經濟學量化「分配不均」的標準。組織內若 Gini > 0.35 表示工作量分布不公平，扣分；< 0.35 時不扣（公平範圍內）。比「標準差」更直觀。

#9 敘述統計 (Mean / Std / Min / Max / Median)

統計 · 基礎

公式 / 邏輯

```
stats.mean(arr) =  $\sum \text{arr}[i] / n$   
stats.std(arr) =  $\sqrt{(\sum (\text{arr}[i] - \text{mean})^2 / (n-1))}$ 
```

實作位置

src/lib/algorithms.ts — stats 物件

說明

基礎統計工具，被各分析模組大量使用。例如員工負載 outlier 偵測用 mean + std。

③ 時間序列 (Time Series)

#10 Exponential Time Decay (指數時間衰減)

時間 · 核心

公式 / 邏輯

```
TIME_DECAY = [1.0, 0.7, 0.5, 0.35, 0.25, 0.15, 0.1, 0.05, 0.02]
weeksAgo = round((asOf - reportWeek) / 7days)
weight = TIME_DECAY[weeksAgo] (9 ■■■■ 0)
```

實作位置

src/lib/algorithms.ts — analyzeEmployeeLoad.getDecayWeight()

說明

員工負載分數依案件離當前時點多遠加權。本週案件權重 1.0, 1 週前 0.7, 2 週前 0.5...
讓近期高強度工作被正確放大。

#11 asOf Snapshot Computation (時點切片)

時間 · v2.1 新增

公式 / 邏輯

```
■■■■■■■■■■ asOf: Date ■■■
analyzeEmployeeLoad(reports, handoffs, emps, asOf)
analyzeBlockerRecord(blocker, history, asOf)
computeHealthSnapshot(asOf, ...)
```

實作位置

src/lib/algorithms.ts, src/lib/orgHealth.ts

說明

v2.1 修正：原本所有分析器內部用 new Date() → 12 週趨勢線每週數字相同。新增 asOf
參數讓「歷史快照」反映該週時點的真實狀態，未來資料 (asOf 之後的) 自動排除。

#12 Local Minima Detection (拐點偵測)

時間 · 信號處理

公式 / 邏輯

```
for i = 1 .. n-2:
  if series[i] < series[i-1] - 3
  AND series[i] < series[i+1] - 3:
    mark as inflection point
```

實作位置

src/lib/orgHealth.ts — detectInflectionPoints()

說明

在 12 週健康度趨勢線上自動標出「V 型谷底」。閾值 3
分避免雜訊，只標真正顯著的低谷。用紅點視覺化，點擊可看當週事件。

#13 Weekly Series Computation (12 週快照序列)

時間 · 衍生

公式 / 邏輯

```
for i = weeks-1 .. 0:
  asOf_i = NOW - i × 7days
  snapshot[i] = computeHealthSnapshot(asOf_i, ...)
```

實作位置

src/lib/orgHealth.ts — computeWeeklySeries()

說明

對過去 12 週分別跑健康度快照，產生時間序列。搭配 Local Minima Detection 提供「整體健康度趨勢線 + 拐點」視覺。

④ 加權評分模型（Weighted Scoring）

#14 Weighted Load Score（員工負載）

加權 · 核心

公式 / 邏輯

```
loadScore = timeWeightedCases × 1.5  
            + blockerLoad × 2.0  
            + mentionsWeighted × 0.8  
            + handoffLoad × 1.0
```

```
level: overload (≥25 ■ P90+) | high (≥15 ■ P75+) | normal (≥6) | low (≥1) |  
idle
```

實作位置

src/lib/algorithms.ts — analyzeEmployeeLoad()

說明

融合 4 種訊號量化「員工真實負荷」。卡點權重最高（×2.0）因為它代表正在燒的工作。用 percentile 而非固定門檻判等級，自動適應公司規模。

#15 ORI — Organizational Risk Index

加權 · 風險

公式 / 邏輯

```
ORI = 0.35 × HCC + 0.25 × DL + 0.25 × BT + 0.15 × CDC
```

```
HCC = ■■■■■■Gini + top1 + outliers■
```

```
DL = ■■■■■■avg■ + ■■■■
```

```
BT = ■■■■■■avgP + P90/P95 ■■
```

```
CDC = ■■■■■■■■■■
```

實作位置

src/lib/algorithms.ts — computeORI()

說明

0-200 反向計分（越低越好）。HCC 權重最高（35%）因為人力集中是組織單點失敗的主因。分 5 級告警：≥175 急、150 注意、125 關注、100 還可以、<100 順利。

⑤ 圖論 / 網絡分析 (Graph Theory)

#19 Directed Weighted Adjacency Matrix (有向加權鄰接矩陣)

圖論 · 核心

公式 / 邏輯

```
for each report r:
  matrix[r.dept][target] += count_mentions(r.text, target)
for each handoff h:
  matrix[h.from][h.to] += 1
```

實作位置

src/lib/algorithms.ts — analyzeDeptNetwork()

說明

從週報文字 + 交接紀錄萃取「誰提到誰」訊號，建構部門互動圖。邊權重 = mention 次數 + 交接量。可分析時間範圍 (4 週、8 週、3 月...)。

#20 Force-directed Graph Layout (力導向佈局)

圖論 · 視覺化

公式 / 邏輯

```
■■■■■ ∝ 1/distance²
■■■■■ ∝ weight × distance
■■■■■■■■■
```

實作位置

src/pages/OrgAnalytics.tsx (SVG 實作)

說明

經典的 force-directed layout 演算法 (類似 D3.js force simulation)。節點大小依「對外溝通總量」，邊粗細依協作密度。讓組織結構視覺化。

#21 Asymmetric Communication Detection (單向溝通偵測)

圖論 · 異常偵測

公式 / 邏輯

```
for each pair (A, B) where A ≠ B:
  if weight(A, B) ≥ 5 AND weight(B, A) = 0:
    mark as one-way communication
```

實作位置

src/lib/algorithms.ts (computeORI.CDC), src/lib/orgHealth.ts (crossDept)

說明

識別「A 部門一直找 B, B 完全沒回應 A」這種組織病徵。計入 ORI 的 CDC 維度與健康度的部門協作維度，每組扣 15-18 分。

⑥ 狀態判定 (State Logic)

#22 Decision Status Helpers (決策狀態動態判定)

狀態 · v2.1 新增

公式 / 邏輯

```
isDecisionOverdueAt(d, asOf):  
  decidedAt ≤ asOf AND dueDate < asOf  
  AND (!completedAt OR completedAt > asOf)  
  
isDecisionInProgressAt(d, asOf):  
  decidedAt ≤ asOf AND NOT overdue AND NOT completed  
  
isDecisionCompletedAt(d, asOf): completedAt ≤ asOf  
daysOverdue(d, asOf): max(0, (asOf - dueDate) / 86400000)
```

實作位置

src/lib/algorithms.ts

說明

v2.1 修正：原本各頁用 d.status === '逾期' 字串判斷，但 orgHealth 用日期判斷 → 兩套標準。新增 4 個 helper 統一邏輯，全系統 (Dashboard / Decisions / Notifications / MeetingPrep / WhatIf / orgHealth / ORI) 共用。自動偵測「dueDate 已過但 status 仍是執行中」的決策。

#23 Risk / Load / Health Level (多層級閾值判定)

狀態 · 分類

公式 / 邏輯

```
Blocker Level: critical (P95+) | high (P90+) | medium (P75+) | normal  
  
Load Level: overload (≥25 ■ P90+) | high | normal | low | idle  
  
Health Level: ■■(85+) | ■■(70+) | ■■■(55+) | ■■■(40+) | ■■■■
```

實作位置

src/lib/algorithms.ts, src/lib/orgHealth.ts (healthLevel)

說明

將連續分數映射到離散層級，方便 UI 配色與排序。每層級對應建議行動文案：critical → 立刻召開協調會議、high → 本週升級處理...

⑦ 預測 / 模擬 (Prediction & Simulation)

#24 What-if Scenario Simulation (情境模擬)

模擬 · v2.1 主功能

公式 / 邏輯

```
scenario = {  
  resolvedBlockerIds: Set<string>  
  expeditedDecisionIds: Set<string>  
  signedHandoffIds: Set<string>  
  extraHeadcount: { [dept]: 0..5 }  
}  
  
shadowData = applyScenario(originalData, scenario)  
projected = computeHealthSnapshot(NOW, shadowData)  
delta = projected.overall - baseline.overall
```

實作位置

src/pages/WhatIf.tsx

說明

互動式 sandbox。對原始資料 fork 一份 shadow data，套用 scenario 修改後，重跑健康度快照，與 baseline 雙圖層比對。回答「假如我這樣做，會怎樣？」demo 神器：教授拉開關就能感受演算法價值。

#25 Smart Suggestion (智能建議文案)

模擬 · 輔助

公式 / 邏輯

```
delta ≥ +5 → ■■■■ ■ ■■■■■■  
delta +2..+5 → ■■■■■■■■  
delta -2..+2 → ■■■■■■■■  
delta -5..-2 → ■■■■■■■■  
delta ≤ -5 → ■■■■ ■■ ■■■■
```

實作位置

src/pages/WhatIf.tsx

說明

把 delta 數字轉成人類可讀的決策建議，降低管理層的解讀成本。

⑧ 附錄：資料保護與工具函式

#* Optimistic UI Sync (樂觀同步)

工具 · 資料層

公式 / 邏輯

```
user action → setState(UI ■■■■)
useEffect ■■ → saveDocumentCollection() → Firestore
syncStatus: idle / syncing / error
```

實作位置

src/hooks/useAppData.ts — syncCollection()

說明

使用者操作立即反映在畫面，背景非同步寫 Firestore。Sidebar 即時顯示同步狀態指示燈。

#* SEED Protection Thresholds (種子資料保護)

工具 · 資料層

公式 / 邏輯

```
reports / handoffs: < 10 ■ → ■ SEED
blockers / history / meetings / decisions / employees: ■■ → SEED
departments / users (v2.1): ■■ → SEED■■■■■■■
■■■ N ■■■■■■ → reset ■■ SEED
```

實作位置

src/hooks/useAppData.ts

說明

防止 Firestore 集合誤刪 / 首次登入時資料為空導致 app 崩潰。v2.1 補強 departments / users 兩個關鍵集合的 fallback。

#* NaN Guards / Clamp (數值安全保護)

工具 · 通用

公式 / 邏輯

```
clamp(v, lo, hi) = max(lo, min(hi, v))
if isNaN(+new Date(str)) → ■■ / ■■ fallback ■■
if denominator > 0 → ■■■■■■ 0 / '■■■■'
submissionRate = Math.min(1, submitted/expected)
```

實作位置

全系統

說明

所有評分函式末段 clamp 進有效區間；所有日期 parse 含 isNaN 檢查；所有除法前驗證分母 > 0。避免 NaN / Infinity 滲透到 UI。

演算法盤點總表

#	演算法	類別	檔案
1	BM25F	資訊檢索	historySearch.ts
2	Robertson-Sparck-Jones IDF	資訊檢索	historySearch.ts
3	多 n-gram Tokenization	資訊檢索	historySearch.ts
4	Substring Boost	資訊檢索	historySearch.ts
5	Synonym Normalization	資訊檢索	historySearch.ts
6	Cosine Similarity (已退役)	資訊檢索	v1 ■■
7	Empirical Percentile	統計	algorithms.ts
8	Gini Coefficient	統計	algorithms.ts, orgHealth.ts
9	Mean / Std / Median	統計	algorithms.ts (stats)
10	Exponential Time Decay	時間序列	algorithms.ts
11	asOf Snapshot	時間序列	algorithms.ts, orgHealth.ts
12	Local Minima Detection	時間序列	orgHealth.ts
13	Weekly Series	時間序列	orgHealth.ts
14	Weighted Load Score	加權評分	algorithms.ts
15	ORI Index	加權評分	algorithms.ts
16	Org Health 6D Score	加權評分	orgHealth.ts
17	Decision Impact Score	加權評分	decisionImpact.ts
18	Leader Scorecard	加權評分	decisionImpact.ts
19	Adjacency Matrix	圖論	algorithms.ts
20	Force-directed Layout	圖論	OrgAnalytics.tsx
21	Asymmetric Detection	圖論	algorithms.ts, orgHealth.ts
22	Decision Status Helpers	狀態判定	algorithms.ts
23	Risk / Load / Health Level	狀態判定	■■
24	What-if Scenario Simulation	預測模擬	WhatIf.tsx
25	Smart Suggestion Text	預測模擬	WhatIf.tsx

共 25 條目 (22 核心演算法 + 3 工具機制) , 分散於約 8 個核心檔案：

src/lib/algorithms.ts (■■■) · src/lib/historySearch.ts · src/lib/orgHealth.ts ·
 src/lib/decisionImpact.ts · src/hooks/useAppData.ts · src/pages/WhatIf.tsx ·
 src/pages/OrgAnalytics.tsx · src/lib/dateUtils.ts

「資料越少，演算法的選擇越重要。」
「跨頁面的一致性，比單頁的炫技更重要。」

—— 串連系統設計哲學